

RoboPet - Care HUD & Decay

Adds visible care stat bars (Hunger / Mood / Energy) at the top of the screen, live decay over time, and low-stat warnings. Code-only, no art needed. Hand directly to Cursor.

Cursor task (build this now)

Implement in order. Adapt field names to the real CareState if they differ.

1. Confirm the model. CareState should have hunger, happiness, energy (ints 0..100), a DateTime lastTick, and bool asleep. Add any missing fields + copyWith.
2. Add decay. Add a pure decayed(state, now) that drains stats by elapsed minutes (slower while asleep; energy recharges while asleep).
3. Update CareNotifier. Run catchUp() in the constructor (offline decay on launch) and a Timer.periodic(30s) tick. Decay first, then add, inside each interaction. Persist to Isar on every change.
4. Build CareHud overlay. Three reactive stat bars at the top, colored by value (green / amber / magenta).
5. Add low-stat warnings. Show a warning line under the bars when any stat is < 20.
6. Wire it up. Register a persistent careHud overlay and add it on game load; keep ref.listen(careProvider, ... dog.applyCareState).
7. Offline catch-up on resume. Use AppLifecycleListener(onResume: ...) to call careProvider.notifier.catchUp().

Acceptance test

- Three bars show at the top and update in real time
- Feed / Pet / Charge instantly raise the matching bar
- Bars slowly decay over time (temporarily raise the rates to verify fast)
- A stat < 20 turns magenta and a warning line appears
- Energy < 20 makes the dog show lowPower (existing applyCareState)
- Close the app for a while, reopen -> bars dropped by the elapsed time (offline decay)
- Values persist across a full restart (Isar)

1) What this adds

The interaction loop already changes care values, but nothing is visible. This makes the pet feel alive: stat bars give instant feedback to Feed/Pet/Charge, decay creates a reason to come back, and warnings drive the core Tamagotchi loop.

2) Decay model

```
// Per-minute decay rates (tune freely)
const double kHungerPerMin = 1.2;
const double kHappyPerMin  = 0.8;
```

```

const double kEnergyPerMin = 1.0;

// Pure function: returns a new state decayed to now.
CareState decayed(CareState s, DateTime now) {
    final mins = now.difference(s.lastTick).inSeconds / 60.0;
    if (mins <= 0) return s;
    final drain = s.asleep ? 0.3 : 1.0; // drains slower while asleep
    return s.copyWith(
        hunger: (s.hunger - kHungerPerMin * mins * drain).round().clamp(0, 100),
        happiness: (s.happiness - kHappyPerMin * mins * drain).round().clamp(0, 100),
        energy: s.asleep
            ? (s.energy + 2.0 * mins).round().clamp(0, 100) // recharges while asleep
            : (s.energy - kEnergyPerMin * mins).round().clamp(0, 100),
        lastTick: now,
    );
}

```

3) CareNotifier (tick + offline catch-up)

```

class CareNotifier extends StateNotifier<CareState> {
    CareNotifier(this._isar) : super(_loadOrDefault(_isar)) {
        catchUp(); // apply offline decay since last session
        _timer = Timer.periodic(const Duration(seconds: 30), (_) => _tick());
    }
    final Isar _isar;
    Timer? _timer;

    void _tick() { state = decayed(state, DateTime.now()); _persist(); }
    void catchUp() { state = decayed(state, DateTime.now()); _persist(); }

    void feed() => _bump(hunger: 30);
    void pet() => _bump(happiness: 20);
    void recharge() => _bump(energy: 40);

    void _bump({int hunger = 0, int happiness = 0, int energy = 0}) {
        final s = decayed(state, DateTime.now()); // decay first, then add
        state = s.copyWith(
            hunger: (s.hunger + hunger).clamp(0, 100),
            happiness: (s.happiness + happiness).clamp(0, 100),
            energy: (s.energy + energy).clamp(0, 100),
            lastTick: DateTime.now(),
        );
        _persist();
    }

    void _persist() { /* write state to Isar */ }

    @override
    void dispose() { _timer?.cancel(); super.dispose(); }
}

```

4) CareHud widget (stat bars)

```

class CareHud extends ConsumerWidget {
    const CareHud({super.key});

    @override
    Widget build(BuildContext context, WidgetRef ref) {
        final s = ref.watch(careProvider);
        final warn = _firstCritical(s);
    }
}

```

```

return Positioned(
  top: 12, left: 12, right: 12,
  child: Column(children: [
    Row(children: [
      _StatBar(label: 'Hunger', value: s.hunger),
      const SizedBox(width: 8),
      _StatBar(label: 'Mood', value: s.happiness),
      const SizedBox(width: 8),
      _StatBar(label: 'Energy', value: s.energy),
    ]),
    if (warn != null)
      Padding(
        padding: const EdgeInsets.only(top: 6),
        child: Text(warn, style: const TextStyle(
          color: Color(0xFFFF3CAC), fontSize: 12,
          fontWeight: FontWeight.bold)),
      ),
  ]),
);
}

String? _firstCritical(CareState s) {
  if (s.energy < 20) return 'Low battery! Charge your pet.';
  if (s.hunger < 20) return 'Your pet is hungry!';
  if (s.happiness < 20) return 'Your pet feels lonely.';
  return null;
}

class _StatBar extends StatelessWidget {
  const _StatBar({required this.label, required this.value});
  final String label;
  final int value;

  Color get _color {
    if (value < 20) return const Color(0xFFFF3CAC); // critical
    if (value < 50) return const Color(0xFFFFB627); // warning
    return const Color(0xFF7CFF6B); // good
  }

  @override
  Widget build(BuildContext context) {
    return Expanded(
      child: Column(crossAxisAlignment: CrossAxisAlignment.start, children: [
        Text('\$label \$value', style: const TextStyle(
          color: Colors.white, fontSize: 11)),
        const SizedBox(height: 3),
        ClipRRect(
          borderRadius: BorderRadius.circular(6),
          child: LinearProgressIndicator(
            value: value / 100.0,
            minHeight: 7,
            backgroundColor: const Color(0x33FFFFFF),
            valueColor: AlwaysStoppedAnimation<Color>(_color),
          ),
        ),
      ]),
    );
  }
}

```

5) Wiring

```
// 1) register persistent overlay:
//   overlayBuilderMap: { 'careHud': (ctx, g) => const CareHud(), ... }

// 2) show HUD + connect care -> dog on game load:
@override
Future<void> onLoad() async {
  overlays.add('careHud');
  ref.listen(careProvider, (_, s) =>
    dog.applyCareState(energy: s.energy, asleep: s.asleep));
  // ...rest of world setup
}

// 3) offline catch-up when app returns to foreground
//   (in the root widget initState):
_lifecycle = AppLifecycleListener(
  onResume: () => ref.read(careProvider.notifier).catchUp(),
);
```

6) Tuning notes

- To test decay fast, temporarily set `kHungerPerMin = 40` (drains in ~2.5 min), then revert.
- Thresholds: < 50 amber, < 20 magenta + warning. Adjust to taste.
- Energy recharges while asleep - ties into a future sleep/wake cycle.
- Bars are reactive via `ref.watch(careProvider)`, so no manual refresh is needed.

Next after this: wire the real dog animation frames so Feed/Pet reactions are actually visible, then a day/night sleep cycle that uses the energy/asleep logic above.